
CSE Web Data Collection

Study and viva guide for the whole team

DA 2009 - Data Collection Methods II

Assignment 2 (Group Project)

University of Colombo

25ada072 - Pasindu | 25ada073 - Sasini | 25ada141 - Salaama | 24ada076 - Nuhan

Website: www.cse.lk

Read the section with your own index number. Then read the shared sections at the front, because any of it can be asked of anybody.

Contents

About this guide	3
How the viva works	3
What the project does	3
The problem we had to solve	3
How we solved it	3
How a request travels through the project	4
Ethical scraping - everybody should know this	4
Who did what	4
24ada076 - Nuhan: Ethical scraping, BeautifulSoup, and reading PDFs	6
Your part, explained simply	6
Files to read before the viva	6
Things you must be able to explain	6
50 questions and answers for 24ada076 - Nuhan	7
25ada072 - Pasindu: Getting the data from the site, and Selenium	14
Your part, explained simply	14
Files to read before the viva	14
Things you must be able to explain	14
50 questions and answers for 25ada072 - Pasindu	15
25ada073 - Sasini: Crawling for documents, and cleaning the data	22
Your part, explained simply	22
Files to read before the viva	22
Things you must be able to explain	22
50 questions and answers for 25ada073 - Sasini	23
25ada141 - Salaama: OCR, saving the files, and the charts	30
Your part, explained simply	30
Files to read before the viva	30
Things you must be able to explain	30
50 questions and answers for 25ada141 - Salaama	31
Quick reference	38
Running each part on its own	38

About this guide

This is for all four of us. It explains what each person built, in plain words, and lists the files you should read before the viva.

Each member has **50 questions with answers** at the end of their section: about 30 on your own part, and 20 on the project as a whole that anybody could be asked.

How the viva works

Part	Time	What happens
Presentation	3 minutes	Any number of us can present the project.
Questions	5 minutes	Questions on what each member contributed.

The assignment says marks are given for **following ethical scraping practices** and for **being able to explain your work**. So knowing why we did something matters as much as knowing what we did.

What the project does

Our group was given the Colombo Stock Exchange, www.cse.lk, and asked to collect data from it. We collect three things:

- **Share prices** - what every listed company traded at today. There are around 290 companies.
- **Company details** - the price, the market value, and the highest and lowest price over the past year, for any company we choose.
- **Announcements** - the notices companies publish, such as dividends and shareholder meetings, and the PDF documents attached to them.

Once we have the data we clean it, check it makes sense, draw a few charts from it, and save it as CSV, Excel and JSON files.

The problem we had to solve

We started the way the course taught us: download the page with `requests`, then read it with BeautifulSoup. On the CSE website that does not work, and it is important that everybody understands why, because it is the reason the whole project is built the way it is.

When we download the page we get about 25,000 bytes of HTML back. But if we ask BeautifulSoup for the words on the page, we get about 24 characters, and if we ask for the tables we get none at all.

The reason is that the CSE website is built with **JavaScript**. The server sends an almost empty page, and then a small program running inside the browser fetches the prices and puts them onto the page afterwards. `requests` only downloads the file. It does not run JavaScript, so it never sees the prices.

This is the difference between **static** and **dynamic** content. A static page has its content sitting in the HTML file. A dynamic page builds its content after loading. CSE is dynamic.

How we solved it

The fix came from the same lecture that described the problem: open the site in a browser, press **F12**, and look at the **Network** tab. That tab lists every request the page makes.

Doing that showed the page fetching its own data from addresses like `https://www.cse.lk/api/tradeSummary`. Those addresses return **JSON**, which is a tidy, structured format that pandas can read directly. One request returns every listed company at once.

So our project reads those addresses instead of trying to read the page. No login is needed, and `robots.txt` does not ask us to stay away from them - it is exactly the same request the page itself makes for any ordinary visitor.

How a request travels through the project

Everything in the project follows the same path. If you can describe these six steps, you can explain how any part of it works.

1. **You click a button** on the screen. `app.py` calls the right function.
2. **The function asks for some data**, for example `api.trade_summary()` in `src/api_client.py`.
3. **That goes to** `Scraper.fetch()` in `src/ethics.py`. Every single request in the project goes through this one function.
4. `fetch()` **applies the rules**: it checks `robots.txt`, waits 1.5 seconds, sends the request saying who we are, and writes a line to `logs/request_log.csv`.
5. **The reply comes back as JSON**, and pandas turns it into a table (a `DataFrame`).
6. **The table goes back to the screen**, and from there it can be cleaned, charted, or saved to a file.

Ethical scraping - everybody should know this

The assignment says marks are given for following ethical scraping practices. This section belongs to 24ada076 - Nuhan, but **every member should be able to answer questions on it**, because it is the part the assignment singles out.

- **robots.txt** is a file websites publish telling automated programs which pages they may visit. Following it is voluntary. We follow it because that is what ethical scraping means.
- **CSE's robots.txt blocks one section**, `/cgi-bin/`, and allows everything else. It does **not** ask us to wait between requests.
- **We wait 1.5 seconds anyway**. Being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server for no benefit to us.
- **We say who we are**. Our User-Agent gives the university, the course and an email address, instead of pretending to be an ordinary browser.
- **We save a copy of every reply**, so we never ask for the same thing twice.
- **We only collect public data**. No logins, no paywalls, no CAPTCHAs, and no personal information about any individual - only company figures that CSE publishes for everyone.
- **We credit the source**. Every file we export names the Colombo Stock Exchange and records when the data was collected.

Who did what

Member	Their part	Files
24ada076 - Nuhan	Ethical scraping, BeautifulSoup, and reading PDFs	src/ethics.py, src/static_scraper.py, src/pdf_extractor.py
25ada072 - Pasindu	Getting the data from the site, and Selenium	src/api_client.py, src/selenium_scraper.py, config.py, warm_cache.py
25ada073 - Sasini	Crawling for documents, and cleaning the data	src/crawler.py, src/cleaner.py, scrapy_project/
25ada141 - Salaama	OCR, saving the files, and the charts	src/ocr_extractor.py, src/storage.py, src/analysis.py

app.py was put together by all four of us. Each tab in the app is labelled with the member whose part it shows.

24ada076 - Nuhan: Ethical scraping, BeautifulSoup, and reading PDFs

The rules every request follows, the first attempt at scraping the page, and getting the text out of PDF documents.

Your part, explained simply

Your part has three pieces.

First, the rules. `src/ethics.py` is the only file in the project that is allowed to talk to the internet. Every request made by anybody's code goes through your `fetch()` function. That function reads `robots.txt`, checks whether we are allowed to request the address, waits 1.5 seconds, sends the request with our honest User-Agent, and writes the result to a log file. Putting it in one place means the rules cannot be forgotten somewhere else.

Second, BeautifulSoup. `src/static_scraper.py` is the ordinary way of scraping - download the page with `requests`, then pick things out of it with BeautifulSoup. On the CSE page it finds almost nothing, and that result is the reason the rest of the project reads the JSON addresses instead. The same file then points BeautifulSoup at the sitemap, which is an ordinary file, and reads it perfectly - proving the library was never the problem.

Third, PDFs. `src/pdf_extractor.py` opens the documents we download and pulls the text out. It uses two libraries: PyPDF2, which is simple and also gives the file details, and pdfplumber, which keeps the layout better and can find tables. It also works out whether a PDF is a real document or a scan, because a scan has no text inside it and has to go to OCR instead.

Files to read before the viva

File	What to look at
<code>src/ethics.py</code>	The whole file. Especially <code>read_robots()</code> , <code>can_fetch()</code> , <code>wait()</code> and <code>fetch()</code> .
<code>src/static_scraper.py</code>	<code>scrape_page()</code> and <code>beautifulsoup_methods()</code> .
<code>src/pdf_extractor.py</code>	<code>page_type()</code> , <code>read_with_pypdf2()</code> and <code>read_with_pdfplumber()</code> .
<code>config.py</code>	Just the settings section - the User-Agent, the delay, and the OCR threshold.

Things you must be able to explain

- What `robots.txt` is, and what CSE's actually says.
- Why we wait 1.5 seconds when nothing forces us to.
- The common HTTP status codes: 200, 403, 404, 500.
- Why `requests` finds almost no text on the CSE page.
- The BeautifulSoup methods and what each one returns.
- The difference between PyPDF2 and pdfplumber.
- Why a scanned PDF gives no text at all.

50 questions and answers for 24ada076 - Nuhan

The first group are on your own part. The later ones are about the project as a whole, and everybody should know them.

EASY - THE BASICS

1. Which files did you write?

`src/ethics.py`, which handles every request the project makes; `src/static_scraper.py`, the requests and BeautifulSoup part; and `src/pdf_extractor.py`, which reads the text out of the PDF documents.

2. In one sentence, what does `src/ethics.py` do?

It sends every web request the project makes, and applies the ethical rules to each one - checking `robots.txt`, waiting between requests, saying who we are, and writing everything to a log.

3. Where does the project read `robots.txt` from?

`https://www.cse.lk/robots.txt`. Every website keeps it at the top level of the site, at that exact name.

4. How often do you read `robots.txt`?

Once at the start of every run, in `read_robots()`. We do not keep a permanent copy, because a site can change its rules and an old copy would mean we were following permissions that had expired.

5. What is a User-Agent?

A short piece of text sent with every request that says what program is asking. Browsers send one that names the browser. Ours names the university, the course, the group and a contact email.

6. What is BeautifulSoup?

A Python library for pulling data out of HTML. You give it the HTML text, it works out the structure, and then you can ask it for tags, text or attributes by name.

7. Which parser did you use with BeautifulSoup?

`html.parser`, which comes with Python so nothing extra has to be installed. For the sitemap we use `xml` mode, which needs `lxml`, and the code falls back to `html.parser` if `lxml` is missing.

8. What does `find()` do, and how is `find_all()` different?

`find()` returns the first tag that matches. `find_all()` returns a list of every tag that matches.

9. Which two libraries do you use to read PDFs?

`PyPDF2` and `pdfplumber`. We use both, because they are good at different things.

10. What are the three types of PDF?

Text based, made by a computer, where the letters are real text. Image based, which is a scan - a picture of the words with no text inside. And hybrid, which has some of each, such as a report with digital tables and a scanned diagram.

11. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

12. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

13. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

14. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

15. What does CSE's robots.txt say?

It blocks one section, `/cgi-bin/`, and allows everything else. It does not set a `Crawl-delay`, so it does not ask us to wait between requests at all.

16. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

17. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

18. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

19. Walk me through what happens when `fetch()` is called.

Four steps, in this order.

First it asks `can_fetch()` whether `robots.txt` allows that address. If not, it stops there and records the attempt as skipped. Second it calls `wait()`, which sleeps until 1.5 seconds have passed since the last request. Third it sends the request through the session, which carries our User-Agent. Fourth it writes a row to `logs/request_log.csv` and checks the status code before returning the data.

```
def fetch(self, url, method="GET", data=None, save_as=None, want="json"):
    allowed, reason = self.can_fetch(url)      # 1. permission
    if not allowed:
        return {"ok": False, ...}
    self.wait()                               # 2. politeness
    response = self.session.post(url, ...)     # 3. the request
    self.log.log_request(method, url, ...)     # 4. the record
```

20. How does `can_fetch()` actually decide?

It uses `RobotFileParser`, which is part of Python's standard library. We give it the text of `robots.txt` with `parse()`, then call `can_fetch(user_agent, url)` and it returns `True` or `False`. We did not write the rule matching ourselves - the standard library already knows the format.

21. Why does `wait()` measure the time instead of always sleeping 1.5 seconds?

Because part of the delay may have passed already. If parsing the last reply took a second, we only need to wait another half second - the server still got its 1.5 seconds of breathing room either way. Sleeping the full amount every time would just make the program slower for no extra politeness.

```
def wait(self):
    since_last = time.time() - self.last_request_time
    if since_last < self.delay:
        time.sleep(self.delay - since_last)
    self.last_request_time = time.time()
```

22. Why does `fetch()` return a dictionary instead of raising an error?

Because it runs behind a web page. If an error were raised and not caught, it would stop the whole app and the screen would go blank. Returning `{"ok": False, "error": ...}` means the tab can show a readable message and everything else keeps working.

23. What is a `requests.Session` and why use one?

A Session keeps the connection to the server open between requests, instead of opening a new one each time. It saves the server a little work on every request, and it lets us set the User-Agent once instead of repeating it on every call.

24. What is in `logs/request_log.csv`?

One row for every request the project has ever sent: the time, whether it was GET or POST, the address, the status code that came back, whether robots.txt allowed it, and the delay we waited. It is the evidence for how the data was collected.

25. What happens if robots.txt cannot be read at all?

We treat every address as blocked. `can_fetch()` returns False with the reason that robots.txt could not be read. If we cannot see the rules we do not guess in our own favour - we stop.

26. Go through the BeautifulSoup methods you used.

`find()` for the first matching tag and `find_all()` for all of them. `select()` and `select_one()` do the same thing but with CSS selectors, which is handy for something like `div.price > span`. `get_text()` gives the words inside a tag, and `strip=True` removes the surrounding whitespace. `get()` reads one attribute, such as the href on a link.

```
soup.find("title")           # first matching tag
soup.find_all("a")           # every matching tag
soup.select_one("div.price")  # CSS selector
tag.get_text(strip=True)     # the words inside
tag.get("href")              # one attribute
```

27. What exactly did `requests` return for the CSE page?

About 25,000 bytes of HTML, with a status code of 200 - so the request itself worked perfectly. But asking BeautifulSoup for the text gave about 24 characters, and asking for tables gave none.

28. How do you know the problem was the page and not your code?

We pointed the same library, with the same methods, at CSE's sitemap - which is an ordinary file with no JavaScript - and it read all 15 pages listed without any trouble. Same tool, working result. That isolates the problem to the page.

29. How would you tell whether a page is static or dynamic before writing any code?

Right-click and choose View Page Source, which shows what the server actually sent. Then right-click and choose Inspect, which shows the page as it is now, after JavaScript has run.

If the numbers appear in Inspect but not in View Source, the page is built by JavaScript and requests will not see them.

30. What is the difference between PyPDF2 and pdfplumber?

PyPDF2 is the simpler one. It extracts text and gives us the file details - the page count, the author, and what the file was created with. It does not keep the layout well and it cannot find tables.

pdfplumber keeps the layout much better and can pull tables out using the position of the text on the page. That is why it is our main method, and we use PyPDF2 alongside it for the file details.

```
# PyPDF2 - simple text and the file details
reader = PdfReader(path)
text = reader.pages[0].extract_text()
details = reader.metadata

# pdfplumber - layout and tables
with pdfplumber.open(path) as pdf:
    text = pdf.pages[0].extract_text()
    tables = pdf.pages[0].extract_tables()
```

31. How does your code decide a PDF needs OCR?

It counts the characters on each page. If a page gives fewer than 50, we treat it as a scan and mark it for OCR.

We use 50 rather than zero because a scan sometimes carries a few stray characters that are not the real document text - a header added afterwards, for instance. Requiring a real amount of text means a handful of stray letters cannot fool us into skipping OCR.

32. Why do you write `page.extract_text() or ""` instead of just `page.extract_text()`?

Because `extract_text()` returns `None`, not an empty string, when there is nothing on the page. Calling `.strip()` or `len()` on `None` would crash. The `or ""` turns it into an empty string so the rest of the code can treat every page the same way.

33. Why did `requests` and `BeautifulSoup` not work on the CSE site?

Because the page is dynamic. `requests` downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

34. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

35. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

36. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

37. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

38. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

39. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

40. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

41. Somebody says the delay makes the project slow. Defend it.

The slowness is the ethics working, not a fault.

Collecting 15 company profiles takes about 25 seconds, and it could take under a second if we fired them all at once. Choosing not to is the whole point - it is the difference between behaving like a considerate visitor and behaving like a denial of service.

It also costs us almost nothing in practice, because the main dataset arrives in a single request. The delay only shows when we deliberately loop over many companies.

42. What did you deliberately choose not to do, and why?

We did not use any technique for getting around a security control. No logins, no paywalls, and no CAPTCHA solving.

We also did not rotate User-Agents or hide who we are. Plenty of scraping tutorials tell you to copy a browser's User-Agent so the site cannot tell you apart from a person. We do the opposite and name ourselves, because the point of the header is to identify the traffic, not to disguise it.

43. How would you prove to the lecturer that you did not overload their server?

By opening `logs/request_log.csv`. It has one row per request with a timestamp, so you can see the requests are spaced at least 1.5 seconds apart, and count them - roughly 30 to 40 for a full run.

The Ethics tab in the app shows the same file on screen.

44. Why did you keep code in the project that does not get the data?

Because it explains the design. Without it, somebody reading the project would reasonably ask why we are not just using BeautifulSoup like everybody else.

`src/static_scraper.py` answers that in one run: it shows the ordinary approach returning almost nothing on this particular site, and then shows the same library working fine on a normal file. It is the reasoning behind the whole project, written as code you can execute.

45. If CSE were a normal static site, what would change?

We would use `src/static_scraper.py` as the main collector instead of the API client. We would find the table in the HTML, loop over the `<tr>` rows, take the `<td>` cells with `get_text(strip=True)`, and build the DataFrame from those.

The ethics layer, the cleaning, the PDF work and the charts would all stay exactly the same, because they do not care where the rows came from.

46. What makes extracting text from PDFs difficult in general?

A PDF is a set of drawing instructions, not a document with a structure. It says where to put each character, not which paragraph it belongs to. So the library has to guess at things a human reads instantly.

Tables are the worst case - they can span pages, have merged cells, and there is nothing in the file saying 'this is a table'. Line breaks are ambiguous, because a break at the edge of the page might be a new paragraph or just the line running out. Headers and footers repeat on every page and you may or may not want them. And formatting like bold, italics and ligatures often comes out wrong.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

25ada072 - Pasindu: Getting the data from the site, and Selenium

Reading the JSON addresses that hold the real data, and driving a real browser when JavaScript has to run.

Your part, explained simply

Your part has two pieces.

First, the data. `src/api_client.py` is where the project actually gets its numbers. The CSE website loads its own data from addresses like `/api/tradeSummary`. Those addresses return JSON - a structured text format - and your file sends the request, takes the reply apart, and turns it into a pandas table. There is one method per thing we want: the market index, the share prices, the sector list, one company's details, and the announcements.

An **API** is just a way for one program to ask another for data. You send a request, you get a reply. A website has one so its own pages can load data without reloading, and it is often left open so other people can use it too.

Second, Selenium. `src/selenium_scraper.py` opens the CSE page in a real browser. Because it is a real browser, the JavaScript runs, the table gets built, and the prices are actually on the page. We then take the finished HTML with `driver.page_source` and read it with BeautifulSoup as normal.

Selenium proves the browser approach works, but it is not our main method. It takes several seconds instead of one, it makes CSE serve the whole page with all its images and scripts, and the table on screen is split into pages so one load only gives about 25 companies. The API gives all of them in one request.

Files to read before the viva

File	What to look at
<code>src/api_client.py</code>	<code>call()</code> , <code>trade_summary()</code> , <code>company_info()</code> and the two helpers <code>get_records()</code> and <code>make_dataframe()</code> .
<code>src/selenium_scraper.py</code>	<code>start_browser()</code> , <code>load_page()</code> and <code>read_table()</code> .
<code>config.py</code>	The <code>ENDPOINTS</code> table - the list of addresses and whether each needs GET or POST.

Things you must be able to explain

- What an API is, in plain words.
- How we found the addresses using the Network tab.
- Why those addresses need POST and not GET.
- How a JSON reply becomes a pandas DataFrame.
- How Selenium makes the JavaScript run.
- What `WebDriverWait` is for and why a fixed sleep is worse.
- Why the API is our main method and Selenium is the backup.

50 questions and answers for 25ada072 - Pasindu

The first group are on your own part. The later ones are about the project as a whole, and everybody should know them.

EASY - THE BASICS

1. Which files did you write?

`src/api_client.py`, which gets the data from CSE; `src/selenium_scraper.py`, which opens the site in a real browser; `config.py`, the settings; and `warm_cache.py`, which saves a copy of the data so the app works offline.

2. What is an API, in plain words?

A way for one program to ask another program for data. You send a request, and you get a reply back, usually as JSON. The lecture compared it to a waiter: you tell the waiter what you want, the waiter goes to the kitchen, and the food comes back.

3. Why do websites have an API?

So their own pages can load data without reloading the whole page, so other developers can build things with their data, and so that automated access does not have to go through the website itself, which is heavier.

4. What is JSON?

A text format for structured data, made of names and values, and lists of those. It is easy for programs to read, which is why almost every API replies in it.

5. Which address gives you the main dataset?

<https://www.cse.lk/api/tradeSummary>. One request to it returns today's trading for every listed company - around 290 of them.

6. What is Selenium?

A tool that controls a real web browser from code. It can open pages, click things, type into boxes and read what is on screen, exactly as a person would.

7. Which browser does your code use?

Microsoft Edge, because that is what is installed on this machine. The code tries Edge first, then Chrome, then Firefox, so it works on anybody's computer.

8. What does headless mean?

The browser runs without showing a window. It is faster and it does not interrupt a demonstration by popping up on screen. Our code can do either.

9. What does `warm_cache.py` do?

It collects everything once and saves a copy of every reply to `data/cache/`. Running it before a demonstration means the app still works if the internet is unavailable, and it also means the demonstration itself sends almost no requests to CSE.

10. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

11. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

12. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

13. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

14. What does CSE's robots.txt say?

It blocks one section, `/cgi-bin/`, and allows everything else. It does not set a `Crawl-delay`, so it does not ask us to wait between requests at all.

15. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

16. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

17. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

18. How did you find these addresses?

We opened the CSE site in a browser, pressed F12 to open the developer tools, opened the Network tab, and reloaded the page. That tab lists every request the page makes while it loads, and among them were the calls the page was making to its own backend.

19. Why do those addresses need POST and not GET?

Because that is what the server accepts. Sending a GET returns an error. It is a little unusual, because the requests only read data and GET would be the normal choice, but it is the server's decision and we found it by trying both.

`config.ENDPOINTS` records which method each address needs, so `call()` picks the right one automatically.

```
ENDPOINTS = {
    "tradeSummary": {"method": "POST", ...},
    "allSecurityCode": {"method": "GET", ...},
}
```

20. Walk me through `call()`.

It takes the short name of an address, looks up whether it needs GET or POST in `config.ENDPOINTS`, builds the full address, and hands it to `Scraper.fetch()`.

It also passes `save_as=endpoint`, which is what gives the saved copy a readable filename such as `data/cache/tradeSummary.json`.

```
def call(self, endpoint, data=None):
    method = config.ENDPOINTS.get(endpoint, {}).get("method", "POST")
    url = f"{config.API_BASE}/{endpoint}"
    return self.scrapers.fetch(url, method=method, data=data,
                               save_as=endpoint)
```

21. Why do you need `get_records()`?

Because the replies do not all have the same shape. Some addresses return a plain list of records. Others wrap the list inside a dictionary under a name such as `reqTradeSummary`. A couple return a list inside another list.

`get_records()` handles all of those and always hands back a plain list, so the rest of the code does not have to care which address it came from.

22. How does a JSON reply become a table?

The reply is a list of dictionaries, where each dictionary is one company. `pd.DataFrame()` accepts exactly that - it makes one row per dictionary and one column per key. So it is a single line once `get_records()` has found the list.

23. How does `company_info()` differ from the other methods?

It sends a value with the request. The address needs to know which company you mean, so we pass `data={"symbol": "SAMP.N0000"}` in the POST. The others take no parameters.

The reply also has several sections, and the company details are inside the one called `reqSymbolInfo`, so we take that part out.

24. What does a symbol like `SAMP.N0000` mean?

`SAMP` is the company - Sampath Bank. `N` means voting shares, where `X` would mean non-voting. `0000` is the share class number. A company can appear more than once if it has different classes of share listed.

25. Why is `several_companies()` slow?

Because each company is a separate request, and every request waits its 1.5 seconds first. Fifteen companies takes a little over 20 seconds.

That is deliberate. It is the only loop in the project that sends many requests in a row, so it is where politeness matters most.

26. What is `to_datetime()` for?

CSE sends dates as the number of milliseconds since 1970 - a number like `1786440420412`. That is unreadable, and worse, pandas would treat it as an ordinary number. `to_datetime()` divides by 1000 and converts it into a real date and time.

27. How does Selenium solve the JavaScript problem?

Because it is a real browser, the JavaScript actually runs. The browser loads the page, the scripts fetch the data, and the table gets built.

We then ask the browser for the finished HTML with `driver.page_source`, which is the page after JavaScript. That is the key difference from `response.text` in the `requests` version.

```
driver.get(url)                # browser loads and runs the page
html = driver.page_source      # the page AFTER JavaScript
soup = BeautifulSoup(html, "html.parser") # then as normal
```

28. What is `WebDriverWait`, and why not just use `sleep`?

When the browser first opens the page the table does not exist yet, because JavaScript is still fetching the data. If we read the HTML immediately we would get the same empty page `requests` gets.

`WebDriverWait` pauses until the element actually appears, then carries on straight away. A fixed `sleep` is worse in both directions - too short and it fails on a slow connection, too long and it wastes time on every run.

```
WebDriverWait(driver, 15).until(
    expected_conditions.presence_of_element_located(
        (By.TAG_NAME, "table")))

```

29. Why is `driver.quit()` inside a finally block?

So it always runs, even if something goes wrong in the middle. If the browser is not closed it keeps running in the background using memory, and after a few failed runs there would be several invisible browsers open.

30. How do you turn the rendered table into a `DataFrame`?

We find the `<table>` with `BeautifulSoup`, take the heading cells from the `<th>` tags, and then loop over the `<tr>` rows taking the `<td>` cells with `get_text(strip=True)`. That gives a list of rows, which `pd.DataFrame()` turns into a table.

If the number of headings does not match the number of cells we number the columns instead, so we never lose the rows.

31. Selenium needs a driver. How is that handled?

Since Selenium 4.6 it handles it itself. Selenium Manager works out which browser is installed, downloads the matching driver the first time it runs, and remembers it. Older tutorials tell you to download `chromedriver` by hand and match the version, which is no longer necessary.

32. Why did `requests` and `BeautifulSoup` not work on the CSE site?

Because the page is dynamic. `requests` downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

33. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

34. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

35. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

36. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

37. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

38. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

39. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

40. Why is using the API better than scraping the page, and is it fair to CSE?

It is better for us because it is faster, more reliable, and gives every company in one request instead of one page at a time.

It is also easier on CSE, which is the part that matters ethically. Loading the page makes their server send the HTML, the JavaScript bundles, the fonts and the images, and then the browser calls that same data address at the end of it anyway. Going straight to the data address skips all of that work.

The Week 6 lecture said it directly: use APIs where available, because they are more reliable and more structured than scraping.

41. What are the limitations of relying on an API?

Rate limits, if the provider sets them. Needing a key or an account for many APIs, although not this one. Only getting the data the provider chose to expose, which may be less than the website shows. And the biggest one for us: an API can change without warning, and ours is not officially documented, so CSE has no reason to tell anybody before changing it.

The way we guard against that is keeping every address in `config.py`, so a change means editing one table.

42. The reply for one company has several sections. How did you know which one to use?

By printing the reply and looking at it. The top level keys were `reqSymbolInfo`, `reqSymbolBetaInfo`, `reqLogo` and `reqTagsLogo`.

The prices and the market value were inside `reqSymbolInfo`, so that is the part we take. The others hold the logo image paths and a volatility figure we do not use.

43. Selenium works. Why is it not your main method?

Three reasons.

It is slow - several seconds against about one for the data address. It is heavy on CSE, because it makes them serve the whole page and all its assets. And it only gets part of the data, because the table on screen is paginated, so one load gives about 25 companies rather than all 290.

We keep it because it proves the browser approach works, and because it would be our fallback if the data addresses ever changed.

44. How would you use Selenium to get all 290 companies?

You would have to work through the pagination - find the next page button, click it, wait for the table to reload, read it, and repeat until the button is disabled, collecting the rows as you go.

That is perfectly doable, but it means one page load per 25 companies, so roughly twelve full page loads with a wait after each. Against a single request that returns everything, it is hard to justify.

45. Why does your Selenium code set the same User-Agent as the rest of the project?

So CSE sees one consistent visitor rather than two different unknown programs. If we identify ourselves honestly in the API requests but let the browser send its default Edge User-Agent, the same project would look like two separate things in their logs, and only one of them would be identifiable.

46. What is the difference between what `requests` returns and what Selenium returns for the same page?

The same address gives about 25,000 bytes with roughly 24 characters of text and no tables through `requests`, and about 140,000 bytes with roughly 4,000 characters of text and a real table through Selenium.

Same URL, same parsing code afterwards. The only difference is that one of them ran the JavaScript.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

25ada073 - Sasini: Crawling for documents, and cleaning the data

Finding and downloading the PDF documents, the Scrapy spider, and tidying the data once it arrives.

Your part, explained simply

Your part has two pieces.

First, crawling. There is a difference between scraping and crawling. Scraping is taking data from a page you already know about. Crawling is finding pages by following links. Most of our project is scraping, because we know the address. Your part is the crawling.

`src/crawler.py` reads the list of circulars, and for each one works out where the PDF actually lives. The list only gives part of the address, something like `upload_report_file/abc123.pdf`, and the PDFs are stored on a different server from the website. We tried the likely addresses until one returned a real PDF, which turned out to be `https://cdn.cse.lk/cmt/` followed by that path. Then it downloads them one at a time, with a pause between each.

`scrapy_project/` does the same job again using Scrapy, which is the framework the lecture described as widely used in industry. The spider starts at the announcement addresses, reads the list, and then follows the link to each PDF - that following step is what makes it a crawler.

Second, cleaning. `src/cleaner.py` fixes the data after it arrives, because data taken from a website is never ready to use straight away. Dates come as huge numbers, some numbers come as text, some columns are empty for every company, and text has extra spaces around it. The file also runs checks afterwards, which is a different job: cleaning fixes the shape of the data, checking asks whether the values are believable.

Files to read before the viva

File	What to look at
<code>src/crawler.py</code>	<code>find_documents()</code> and <code>download_documents()</code> .
<code>src/cleaner.py</code>	<code>clean()</code> and <code>check()</code> , and the <code>describe()</code> counts.
<code>scrapy_project/cse_crawler/spiders/cse_announcements.py</code>	<code>start()</code> , <code>parse()</code> and <code>parse_document()</code> .
<code>scrapy_project/cse_crawler/settings.py</code>	The polite settings near the top.

Things you must be able to explain

- The difference between scraping and crawling.
- How the full PDF address is built from the fragment.
- How we check a downloaded file really is a PDF.
- How the Scrapy spider follows links.
- Which Scrapy settings keep it polite.

- The four things that had to be cleaned in the CSE data.
- The difference between cleaning and checking.

50 questions and answers for 25ada073 - Sasini

The first group are on your own part. The later ones are about the project as a whole, and everybody should know them.

EASY - THE BASICS

1. Which files did you write?

`src/crawler.py`, which finds and downloads the PDF documents; `src/cleaner.py`, which tidies the data and checks it; and the Scrapy project in `scrapy_project/`.

2. What is the difference between scraping and crawling?

Scraping is taking data from a page you already know about. Crawling is finding pages by following links from one to the next.

Most of our project is scraping, because we know the address. My part is the crawling - we do not know which documents exist until we look.

3. Why are crawlers sometimes called spiders?

Because they crawl across the web the way a spider moves across its web. They start from a few known addresses and follow links outward, building up a network of pages they have found.

4. What is Scrapy?

A Python framework built specifically for crawling and scraping. It handles the queue of requests, the delays, robots.txt and saving the output, so you only write the part that says what to extract and which links to follow.

5. Why does scraped data need cleaning at all?

Because it comes out in whatever shape the website happened to send it, not in a shape that is ready to analyse. The course listed 'data collected through scraping is ready for analysis' as a misconception.

6. What are the four things you had to clean in the CSE data?

Dates arriving as very large numbers. Numbers arriving as text. Columns that are empty for every single company. And text with extra spaces around it.

7. Why does the crawler skip documents you already have?

Because downloading the same file twice costs CSE bandwidth and gains us nothing. Before each download we look in `data/pdfs/` for a file already saved under that document id, and if it is there we move on.

8. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

9. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, `www.cse.lk`. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

10. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

11. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

12. What does CSE's robots.txt say?

It blocks one section, /cgi-bin/, and allows everything else. It does not set a Crawl-delay, so it does not ask us to wait between requests at all.

13. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

14. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

15. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

16. Walk me through your crawler.

Two stages.

`find_documents()` asks for the list of circulars, and for each one builds the full address of the PDF. It also removes duplicates, because the same document can appear twice and downloading it twice would waste CSE's bandwidth for nothing.

`download_documents()` then fetches them one at a time through the ethical session, so each download waits its 1.5 seconds. It skips anything we already have on disk.

17. How did you work out where the PDFs are stored?

The list only gives part of the address, something like `upload_report_file/abc123.pdf`, which is not a working link on its own. And the PDFs are not on the website itself - they are on a separate server.

So we tried the likely addresses until one returned a real PDF. `www.cse.lk/cmt/...` gave 404, `cdn.cse.lk/...` gave 403, and `cdn.cse.lk/cmt/...` returned 200 with content type `application/pdf`.

18. How do you check a downloaded file really is a PDF?

Every PDF file starts with the four characters `%PDF`. We check the first four bytes before saving. If they do not match, something else came back - an error page, for instance - and we do not save it as a PDF.

```
if not content[:4] == b"%PDF":
    results.append({"Title": title, "Result": "Not a PDF file"})
    continue
```

19. Why limit how many documents you download?

There are far more documents on the site than we need to show the technique works. Downloading all of them just because we could would put load on their server for no benefit. The lecture put it as: only scrape the data you need.

20. Explain the Scrapy spider.

`start()` sends POST requests to the two announcement addresses, because CSE does not answer GET. `parse()` reads the JSON reply, builds the full address of each PDF, and yields a new request for it - that yielding of a new request is the crawling step. `parse_document()` then records what came back at the other end.

```
def parse(self, response, record_name, label):
    reply = json.loads(response.text)
    for record in reply.get(record_name, []):
        pdf_url = self.CDN_BASE + record["path"]
        yield scrapy.Request(pdf_url,                # follow the link
                             callback=self.parse_document,
                             method="HEAD")
```

21. Why does the spider use HEAD instead of GET?

A HEAD request asks only for the details of the file - its type and its size - without downloading the file itself. We want to confirm each document exists and see how big it is, and downloading several megabytes of PDF just to find that out would be wasteful.

22. Which Scrapy settings keep the spider polite?

`ROBOTSTXT_OBEY = True`, which makes Scrapy check robots.txt itself. `DOWNLOAD_DELAY = 1.5`, matching the rest of our project. `CONCURRENT_REQUESTS = 1`, where the default is 8 - that would mean eight connections at once. `AutoThrottle`, which slows down further if the server starts responding slowly. And `CLOSESPIDER_ITEMCOUNT = 40` so it stops after enough items.

23. Why does the spider record None rather than 0 when the file size is missing?

Because 0 is a measurement, and a wrong one - it says the file is empty when it is not. None honestly says we did not find out.

The CSE server does not always send the Content-Length header back on a HEAD request. Writing 0 in that case would put false figures into our dataset, which is exactly the kind of quiet data quality problem the course warned about.

24. Explain the date problem.

CSE sends dates as the number of milliseconds since 1 January 1970 - a number like 1786440420412. Left alone, pandas treats that as an ordinary number, which means you could sort by it, or worse, calculate an average of it, and nothing would warn you.

We convert those columns with `pd.to_datetime(column, unit="ms", utc=True)`, and then into Colombo time. The numbers are counted in UTC, and Sri Lanka is 5 hours 30 minutes ahead - without that step every trade would look as if it happened before the market opened.

25. How do you avoid converting a column by mistake?

Three checks, and all of them have to pass. The column name has to contain a word like 'date', 'time' or 'created'. The column has to hold numbers. And the values themselves have to be larger than a trillion, which any real date in milliseconds will be but a price or a quantity never is.

```
if not any(word in column.lower()
           for word in ("date", "time", "created")):
    continue
if not pd.api.types.is_numeric_dtype(tidy[column]):
    continue
...
if not values.empty and values.median() > 1_000_000_000_000:
    tidy[column] = (pd.to_datetime(tidy[column], unit="ms", utc=True,
                                   errors="coerce")
                   .dt.tz_convert("Asia/Colombo")
                   .dt.tz_localize(None))
```

26. Why does a number stored as text matter?

Because you cannot add up text, and sorting it sorts alphabetically. That puts "9" after "100", because it compares the first character. Any chart or total built on it would be wrong.

27. How do you decide a text column is really numbers?

We take a sample of the column, try converting it to numbers, and see how much of it succeeds. If more than 90 per cent converts, it was a number column stored as text and we convert the whole thing.

We do not require 100 per cent, because a genuine number column can still have a few blanks or dashes in it.

28. What is the difference between cleaning and checking?

Cleaning fixes the shape of the data - the types, the empty columns, the duplicates. Checking asks whether the values are believable.

A share price of minus five rupees would be perfectly tidy and completely wrong. Cleaning would never catch it; a check does.

29. What checks do you run?

That there are rows at all. That no price is negative. That nothing moved by more than 100 per cent in a day, which would be suspicious. That every row has a trading symbol. That no company appears twice. And an overall completeness figure - what percentage of cells actually have a value.

30. Why do you copy the DataFrame before cleaning it?

So the original stays untouched. If the cleaning turns out to have a bug, we can look at the raw version and compare, without having to go back to CSE and download everything again.

31. How do you show that the cleaning did something?

We count the problems before and after and show them side by side - the number of rows, empty cells, empty columns, duplicate rows, number columns and date columns. Saying 'we cleaned the data' means very little; showing date columns going from 0 to 1 is evidence.

32. Why did requests and BeautifulSoup not work on the CSE site?

Because the page is dynamic. requests downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

33. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

34. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

35. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

36. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

37. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

38. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

39. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

40. Your first version of the spider collected nothing. What happened?

It reported 0 items and finished in about a hundredth of a second, without sending a single request.

The method for sending the first requests used to be called `start_requests()`, which is what the course notes use. Scrapy 2.13 replaced it with an asynchronous `start()` method, and by version 2.17, which is what we have installed, `start_requests()` was removed completely. So Scrapy never called our method and nothing warned us.

It is a real example of a scraper breaking because something underneath it changed - usually that is the website, but here it was the library.

41. Why does the project have two crawlers doing the same job?

`src/crawler.py` is ours, written directly with `requests`, and it is what the app actually uses. It is simple and it goes through the same ethical layer as everything else in the project.

The Scrapy version shows the framework approach that the course taught and that industry uses. It runs as a separate process, because Scrapy's engine cannot be started twice inside one program.

42. How would you extend the crawler to follow links deeper?

At the moment we go two levels: the listing, then each document. To go deeper you would have `parse_document()` yield further requests for any links it found, and Scrapy would queue those the same way.

You would also need to keep a set of addresses already visited so it cannot loop forever, and set `DEPTH_LIMIT` so it does not wander across the whole site. `allowed_domains` already stops it leaving `cse.lk`.

43. Your before and after table shows the number columns going down by one. Why is that a good thing?

Because that column became a date column instead. It was being counted as a number, which is exactly the problem - it was a timestamp being treated as an ordinary figure.

So one column moved out of the number count and into the date count. The two changes together are the fix showing up in the numbers.

44. Is dropping empty columns always the right thing to do?

Not always, and it is worth being honest about. A column that is empty today might be filled in tomorrow, or might be empty only for the particular set of companies we happened to fetch.

We do it because those columns are noise on screen and in every exported file, and because we keep the raw version, so nothing is actually lost. If we were building something that ran every day, dropping columns based on one day's data would be the wrong call.

45. How would you handle a company that legitimately moved more than 100 per cent?

The check does not delete anything - it flags it. That is the important part of the design.

A big move can be real, after a share split or a return from suspension. So the check says 'look at this', not 'this is wrong'. Automatically deleting rows that look odd is how you quietly lose real data.

46. If you had more time, what would you add to the cleaning?

Two things. Cross-checking against another source, so we could tell whether a strange number is a collection error or a real market event. And keeping the data from several days, which would let us spot a value that is impossible given yesterday's - a price that has jumped a hundred times overnight, for example.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, `robots.txt` followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

25ada141 - Salaama: OCR, saving the files, and the charts

Reading scanned documents that contain no text, saving everything with the source credited, and the charts.

Your part, explained simply

Your part has three pieces, and the first is the hardest thing in the project.

First, OCR. Some of the CSE documents are scans - somebody printed a page, signed it, and put it through a scanner. What is inside the file is a photograph of the words. There is no text in it, so no PDF library can pull anything out, no matter which one you try.

OCR stands for Optical Character Recognition, and it means getting a computer to look at a picture of writing and work out the letters. `src/ocr_extractor.py` does it in four steps: turn the PDF page into an image, clean the image up with OpenCV, read the letters with pytesseract, and then tidy the result into a table with pandas.

The cleaning up matters more than people expect. Tesseract reads plain black text on a white background best, so we remove the colour and then turn every pixel either black or white. There is also the lecture's optional noise removal step, which at the lecture's setting leaves the image unchanged.

Second, saving. `src/storage.py` writes the data out as CSV, Excel and JSON. Every file carries the source and the time it was collected, inside the file itself. Each format needs a different trick: comment lines at the top of a CSV, a second sheet in an Excel file, and a details section in a JSON file.

Third, the charts. `src/analysis.py` builds the bar charts and works out the summary figures - how many companies rose, how many fell, and the total turnover.

Files to read before the viva

File	What to look at
<code>src/ocr_extractor.py</code>	The whole file. Especially <code>preprocess_image()</code> and <code>ocr_pdf_page()</code> .
<code>src/storage.py</code>	<code>export()</code> - look at how each format carries the source.
<code>src/analysis.py</code>	<code>chart_movers()</code> and <code>market_figures()</code> .

Things you must be able to explain

- What OCR is and why we needed it here.
- The four steps of the OCR pipeline.
- Why the image is turned grey and then black and white.
- What the dpi setting changes.
- How we check whether Tesseract is really installed.
- Why OCR makes mistakes, and how we spot them.
- How the source gets into each type of exported file.

50 questions and answers for 25ada141 - Salaama

The first group are on your own part. The later ones are about the project as a whole, and everybody should know them.

EASY - THE BASICS

1. Which files did you write?

`src/ocr_extractor.py`, which reads scanned documents; `src/storage.py`, which saves the data as CSV, Excel and JSON; and `src/analysis.py`, which builds the charts and the summary figures.

2. What does OCR stand for, and what does it do?

Optical Character Recognition. It means getting a computer to look at a picture of writing and work out which letters it is seeing, so the words become text you can search and copy.

3. Why did this project need OCR at all?

Because one of the CSE circulars is a scan. Somebody printed the page, signed it and put it through a scanner, so the file contains a photograph of the words. Ordinary PDF extraction returns nothing from it, no matter which library you use.

4. Name some real uses of OCR outside this project.

Digitising invoices, receipts and bank statements. Reading number plates for traffic cameras and car parks. Scanning passports at airports. Reading insurance claim forms. And reading text aloud for people who cannot see well.

5. Which three libraries make up your OCR pipeline?

PIL (Pillow) for opening and handling the image, OpenCV for cleaning the image up, and pytesseract for actually reading the letters. pandas then tidies the result.

6. Which formats do you save the data in?

CSV, Excel and JSON. CSV opens anywhere, Excel is what most people actually use, and JSON keeps the structure for another program to read.

7. What do your charts show?

Three things: the companies whose price moved the most today, the companies with the highest turnover, and how each sector of the market moved.

8. Why do you save the image after each cleaning step?

So they can be shown side by side in the OCR tab - the grey version, the black and white version, and the version after the noise step. It makes the preprocessing visible instead of something that just happens invisibly before the text appears.

9. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

10. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

11. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

12. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

13. What does CSE's robots.txt say?

It blocks one section, /cgi-bin/, and allows everything else. It does not set a Crawl-delay, so it does not ask us to wait between requests at all.

14. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

15. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

16. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

17. What are the four steps of the OCR pipeline?

Image acquisition - turn the PDF page into an image.

Preprocessing - improve the image so the text is easier to read: remove the colour, then turn it pure black and white.

Text recognition - pytesseract reads the letters.

Post-processing - tidy the text into lines with pandas, and save it.

18. How do you turn a PDF page into an image?

With pdfplumber's `to_image()`, at 200 dpi, and then save it as a PNG. OCR works on pictures, not on PDFs, so the page has to become an image before anything else can happen.

```
with pdfplumber.open(pdf_path) as pdf:
    page_image = pdf.pages[page_number - 1].to_image(resolution=dpi)
    page_image.save(str(image_path))
```

19. Why convert the image to grey?

Because colour tells us nothing about which letter something is. A grey image has one brightness value per pixel instead of three colour values, so there is less for the OCR engine to work through and nothing useful has been lost.

20. What is thresholding, and why do you use the adaptive kind?

Thresholding turns every pixel either pure black or pure white, so the letters have hard edges instead of soft grey ones. That is what Tesseract reads best.

We use adaptive thresholding, which works out the cut-off separately for each small part of the page instead of using one value for the whole thing. That matters on a scan, where one side of the page is often darker than the other - a single cut-off would lose the text on the dark side.

```
thresh = cv2.adaptiveThreshold(  
    grey, 255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    31, 2)
```

21. What does the noise removal step do?

In our project, nothing - and it is worth knowing why. It is the optional step from the lecture's OCR example, which uses a 1x1 kernel. A 1x1 kernel leaves every pixel as it was; we checked, and 0 pixels changed.

The idea behind the step is to remove specks left by the scanner, which OCR would otherwise read as stray full stops. But those specks are black dots on a white page, and MORPH_OPEN removes small white spots, so doing it properly would need MORPH_CLOSE with a 2x2 or 3x3 kernel. A bigger kernel can also rub out thin parts of letters, so it would need testing before we changed it. We kept the lecture's setting.

```
kernel = np.ones((1, 1), np.uint8)      # 1x1: changes nothing  
cleaned = cv2.morphologyEx(threshold, cv2.MORPH_OPEN, kernel)
```

22. What does the dpi setting change?

How large the image is. dpi means dots per inch, so a higher number means more detail for the OCR engine to work with, but a bigger image and a slower job.

At 72 dpi small print turns to mush and accuracy collapses. We use 200, which is enough for a printed document. 300 helps with very small print but takes noticeably longer.

23. Why do you convert between OpenCV and PIL?

Because the libraries hold images differently. OpenCV works with numpy arrays - grids of numbers - while PIL has its own image type, and the lecture passed PIL images to pytesseract. `Image.fromarray()` converts the OpenCV result into a PIL image, so we can use OpenCV for the cleaning and pytesseract for the reading.

To be accurate: pytesseract will also take a numpy array directly. We convert anyway so the code follows the lecture's pipeline, where each library gets the kind of image it is normally used with.

24. How do you know whether Tesseract is actually installed?

Importing pytesseract is not enough. It is only a small Python wrapper - the real work is done by the Tesseract program, which on Windows has to be installed separately. The import succeeds even when the program is missing.

So we call `pytesseract.get_tesseract_version()`, which forces it to go and find the program. If that raises an error, we know it is not there and we show the install instructions instead of a crash.

```
def tesseract_ready():  
    try:  
        version = pytesseract.get_tesseract_version()  
        return True, f"Tesseract {version} is installed and ready."  
    except Exception:  
        return False, INSTALL_HELP
```

25. What is the difference between `image_to_string` and `image_to_data`?

`image_to_string` gives you the text as one block. `image_to_data` gives you far more - every word separately, where it sits on the page, and a confidence score from 0 to 100 saying how sure Tesseract is. We use both: the string for reading, and the data so we can show which words are least certain.

26. Why only OCR some pages instead of the whole document?

Because OCR is slow and it guesses, while reading a text layer is instant and exact. The PDF module tells us which pages returned nothing, and we run OCR only on those. Running it over pages we already read perfectly would waste time and introduce mistakes into text that was already correct.

27. How does the source get into each type of file?

Each format needs a different trick.

A CSV gets comment lines at the top, each starting with `#`. Excel shows them, and pandas can skip them when reading the file back with `pd.read_csv(path, comment="#")`.

An Excel file gets a second worksheet called Source, because a spreadsheet has no way to hold a comment.

A JSON file gets a details section above the records.

```
with open(path, "w", encoding="utf-8-sig", newline="") as f:
    f.write(f"# {config.ATTRIBUTE}\n")
    f.write(f"# Collected on: {collected}\n")
    df.to_csv(f, index=False)
```

28. Why does crediting the source matter?

Because a spreadsheet passed on to somebody else with no source attached is how data gets misused. Somebody finds the file six months later and has no idea where the numbers came from or when they were true.

Putting it inside the file means it travels with the data, instead of living in an email that gets lost.

29. What does `utf-8-sig` mean when writing the CSV?

It writes a small marker at the start of the file that tells Excel the file is UTF-8. Without it, Excel on Windows sometimes guesses the wrong encoding and company names with special characters come out as nonsense.

30. Why do you strip odd characters out of the PDF filenames?

Because the announcement titles contain characters Windows will not accept in a filename, such as slashes and quotation marks. We keep only letters, numbers, spaces, dots, underscores and dashes, and cut the name to a sensible length so the full path does not get too long.

31. Why is the movers chart a different shape from the turnover chart?

Because they answer different questions. The movers chart shows direction as well as size, so it spreads out both ways from a zero line, with risers on one side and fallers on the other.

Turnover has no direction - it is only ever an amount - so it is a straightforward bar chart in one colour. Adding a second colour there would suggest a difference that does not exist.

32. Why green and red, and why the extra labels?

Green up and red down is the usual convention for share prices, so it is what anyone reading a market chart expects.

But red and green are the hardest pair to tell apart for people with colour blindness, so every bar also carries the direction in words and a plus or minus sign. The colour is never the only thing telling you which way the price went.

33. Why show turnover in millions rather than the raw number?

Because the raw figures run into the billions of rupees, and an axis labelled 1500000000 is unreadable. Dividing by a million gives numbers a person can compare at a glance, and the axis title says the unit.

34. What is `cliponaxis=False` for?

Without it, Plotly cuts off any label that reaches past the edge of the plot. We put the value at the end of each bar, so the longest bar's label was being clipped. Turning clipping off lets the label draw over the margin, and we also widen the axis range to leave room.

35. Why did `requests` and `BeautifulSoup` not work on the CSE site?

Because the page is dynamic. `requests` downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

36. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

37. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

38. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

39. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

40. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

41. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

42. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

43. Is OCR accurate? Show that you understand its limits.

Not completely, and the output shows it. It runs words together where the spacing is tight, and it misreads logos and stamps as letters because it is looking for letter shapes everywhere.

That is why it is the last resort rather than the first choice. If a document has a real text layer we always read that instead, because it is exact. We also record the confidence score for every word, so anything the engine was unsure about can be checked by hand rather than trusted silently.

44. What are the main challenges in OCR, and how do you deal with them?

Poor image quality - dealt with by scanning at a higher resolution and improving the contrast, which is what our preprocessing does.

Complicated layouts with columns and tables, where the engine has to work out the reading order.

Handwriting, which needs a different kind of model altogether.

Unusual fonts and symbols. And pages that are slightly rotated, which need straightening before anything else, because Tesseract expects roughly horizontal lines.

45. If OCR gave poor results on a document, what would you try?

Raise the dpi first, since small print is the most common cause. Then look at the preprocessed image - if the thresholding has eaten thin strokes or left the background patchy, adjust the block size in the adaptive threshold.

After that, crop to just the part of the page that matters, so the engine is not trying to read a logo or a signature. And if the page is tilted, straighten it, because a few degrees of rotation hurts accuracy more than people expect.

46. What do the summary figures actually tell you about the market?

They give the breadth of the day rather than just the headline. The index can rise while more companies fall than rise, which happens when a few large companies pull the average up.

So we count how many rose, how many fell and how many did not move, alongside the index. If more fell than rose, the day was broadly negative even if the index went up - and the written summary says that in plain words rather than leaving it to the reader.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. robots.txt does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

Quick reference

Thing	Value
Website	https://www.cse.lk
Text requests finds on the page	about 24 characters, no tables
Companies in one request	around 290
Delay between requests	1.5 seconds
Section blocked by robots.txt	/cgi-bin/ only
Crawl-delay CSE asks for	none
Main data address	/api/tradeSummary
Where the PDFs live	https://cdn.cse.lk/cmt/
PDF libraries	PyPDF2 and pdfplumber
OCR libraries	PIL, OpenCV, pytesseract
OCR resolution	200 dpi
A page is a scan if it has under	50 characters
Export formats	CSV, Excel, JSON
Request log	logs/request_log.csv

Running each part on its own

```
python -m src.ethics          # 24ada076 - Nuhan
python -m src.static_scraper  # 24ada076 - Nuhan
python -m src.pdf_extractor   # 24ada076 - Nuhan
python -m src.api_client      # 25ada072 - Pasindu
python -m src.selenium_scraper # 25ada072 - Pasindu
python -m src.crawler         # 25ada073 - Sasini
python -m src.cleaner         # 25ada073 - Sasini
python -m src.ocr_extractor    # 25ada141 - Salaama
python -m src.storage         # 25ada141 - Salaama
python -m src.analysis        # 25ada141 - Salaama
```

Data source: Colombo Stock Exchange (www.cse.lk). Collected for academic purposes only. This is one trading day. It is not investment advice.